

Assignment Report

Task-1

Baseline Results

Metric	Score
Accuracy	0.8152
ROC-AUC	0.7949

Confusion matrix:
[[13065 1842]
[2571 6400]]

For our baseline model, we achieved an accuracy of roughly 81.5% and an ROC-AUC of 0.79. This is a pretty good starting point considering we just filled missing values with the median and did basic label encoding. However, looking at the confusion matrix, the model completely missed 2,571 actual cancellations (false negatives). This shows that while the raw data has predictive power, we need feature engineering to help the model catch those hidden cancellation patterns.

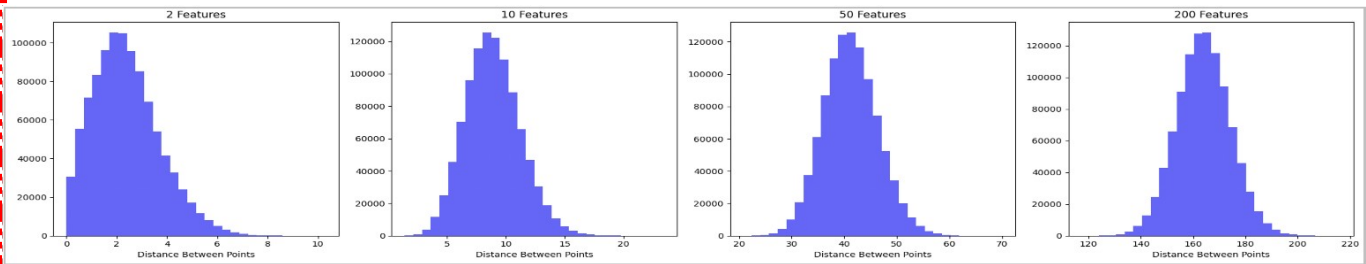
What is a feature?

A feature is an individual, measurable property or characteristic of a phenomenon being observed. In tabular data, it represents a single column that provides information to the machine learning model.

A **good feature** is one that has a strong, logical relationship with the target variable, such as 'lead_time'; how far in advance a customer books heavily influences their likelihood of canceling.

A **bad feature** is overly unique, noisy, or irrelevant, such as a raw 'reservation_status_date'. Giving a model a raw date string provides no mathematical pattern for it to learn from and can lead to overfitting.

Task-2

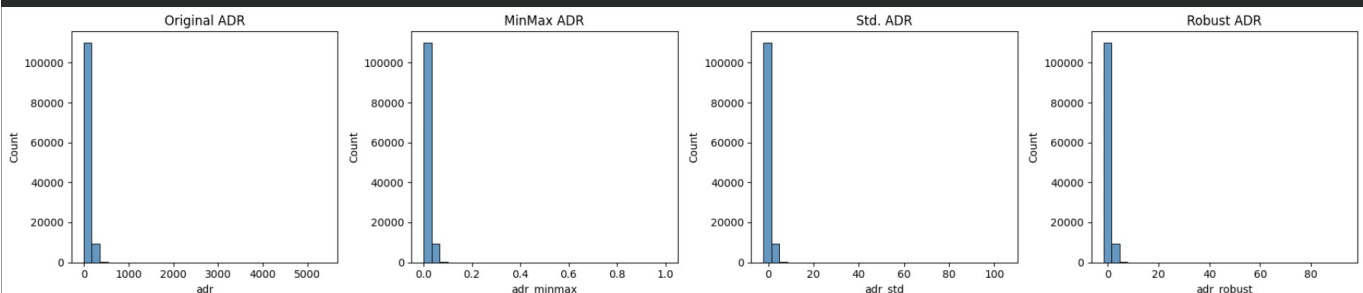


As the number of dimensions increases from 2 to 200, the distribution of distances shifts to the right and becomes much narrower. In low dimensions, there is a clear distinction between points that are "close" and points that are "far." However, in high-dimensional space, the data becomes incredibly sparse. The difference between the nearest neighbour and the farthest neighbour shrinks to almost zero, meaning every point appears equally far away from every other point. This "Curse of Dimensionality" makes learning exceptionally hard for distance-based algorithms like KNN because it destroys the mathematical concept of proximity. This proves the absolute necessity of feature engineering and selection: we must compress and select only the most vital features to keep the data dense and the distances meaningful.

Task-3

Summary Statistics Change

	Original ADR	MinMax ADR	Std. ADR	Robust ADR
Mean	101.8298	0.0200	0.0000	0.1291
Std	50.5348	0.0093	1.0000	0.8910
IQR	56.7200	0.0105	1.1224	1.0000



For this dataset, **RobustScaler** is definitively the best choice. The original hotel data (**adr** and **lead_time**) contains extreme outliers, such as a single booking with an ADR of over \$5,000. *StandardScaler* relies on the mean, and *MinMaxScaler* relies on the absolute maximum value. Both of these are severely distorted by outliers, causing the majority of normal bookings to be squashed into a tiny, unreadable range. *RobustScaler* ignores these extremes by using the Median and the

Interquartile Range (IQR), successfully centering the bulk of our normal data while keeping the outliers from ruining the scale.

Task-4

```
Unscaled KNN-euclidean accuracy: 0.7496
Standard-Scale KNN-euclidean accuracy: 0.7500
Robust-Scale KNN-euclidean accuracy: 0.7519
Robust-Scale KNN-manhattan accuracy: 0.7522
```

Why scaling changes learning??

Scaling fundamentally changes how the model learns. Without scaling, a feature with large numbers completely dominates the distance calculation over a feature with small numbers. Scaling forces all features to contribute equally. Because (KNN) relies heavily on calculating the physical distance between data points, leaving the data unscaled (0.7496) allowed features with larger numerical ranges (*lead_time*) to unfairly dominate the distance calculations. Applying *StandardScaler* brought all features onto a level playing field, resulting in a slight improvement (0.7500).

Outliers Sensitivity:

The jump from *StandardScaler* to *RobustScaler* (0.7519) perfectly demonstrates the dataset's sensitivity to extreme outliers. *StandardScaler* relies on the mean and standard deviation, which are easily warped by extreme hotel prices or massive lead times. *RobustScaler* ignores these extremes by using the median and Interquartile Range (IQR), proving to be the most effective way to scale this specific dataset without squashing the normal data points.

Finally, switching the distance metric from Euclidean to Manhattan yielded the highest overall accuracy (0.7522). While Euclidean calculates the direct "straight-line" distance, Manhattan calculates the absolute grid-based distance. In high-dimensional spaces with varying feature types, Manhattan distance often proves to be slightly more stable and less sensitive to remaining edge-case variations.

Task-5

```
Pipeline cross-validation Accuracy: 0.6527
```

The end-to-end pipeline successfully automated the imputation, power transformation (Yeo-Johnson), and standard scaling of our numeric features (*lead_time*, *adr*, *stays_in_weekend_nights*), passing them directly into a Logistic Regression classifier. Achieving a cross-validation accuracy of 65.27% is a solid foundation, but it importantly highlights the limits of raw numeric data. Even with

perfect, leak-free preprocessing, using only three basic numeric columns is not enough to capture the complex human behaviour behind hotel cancellations therefore we need to add more features to give the model the context it needs for higher accuracy.

Task-6,7 combined

5 extracted features from data to provide deeper context to model:

month & year: Extracted from the **reservation_status_date**. These capture long-term macro trends and annual seasonality (summer vs. winter booking behaviors).

day_of_week: Extracted from the **reservation_status_date**. This tracks administrative behaviour, capturing whether cancellations are more likely to occur on specific weekdays (middle of the work week).

is_weekend_arrival: A binary flag extracted from the arrival date. This acts as a proxy to differentiate between leisure travellers (who typically arrive on weekends and may cancel due to personal reasons) and business travellers (who arrive on weekdays with stricter schedules).

meal & customer_type: Created via *One-Hot Encoding*. By breaking these categories into distinct binary columns, the model can assess the financial commitment level of specific packages.

8 constructed features from data to provide deeper context to model:

price_per_person (Ratio): Divides the total room price (ADR) by the number of guests. This normalises the cost to reveal if the customer is a budget traveler or a luxury guest, which strongly impacts price sensitivity and cancellation risk.

special_req_ratio (Ratio): Divides the total special requests by the number of weeknights. This measures the customer's level of engagement and investment in the trip per day.

total_nights (Interaction): Combines weekend and weeknights to capture the overall duration of the stay, as longer trips often require more planning and have different cancellation probabilities than short weekend stays.

total_revenue_potential (Interaction): Multiplies ADR by total nights to find the total financial value of the booking. High-value bookings might be treated differently by customers compared to cheap, single-night stays.

avg_adr_by_market (Aggregated): Maps the historical average room price based on the customer's market segment. This provides a baseline expectation of what a specific type of customer (Corporate vs. Direct) typically spends.

avg_price_by_room (Aggregated): Maps the historical average price based on the specific room type reserved, giving the model context on the baseline value of the physical asset being booked.

lead_time_squared (Polynomial): Squares the lead time to emphasise extreme values. This helps the model capture non-linear relationships, as the risk of cancellation compounds significantly the further out a booking is made.

special_requests_squared (Polynomial): Squares the total special requests to exponentially weight highly engaged customers, as customers making multiple specific demands are highly invested and historically less likely to cancel.

Avoiding Leakage:

1-Aggregation Leakage Prevention: When calculating the grouped averages (avg_adr_by_market and avg_price_by_room), I calculated the .mean() strictly using only the training dataset (Xtrain). I then mapped these training averages onto the dataset. If I had calculated the average using the entire dataset, information from the test set would have leaked into the training features.

2-Handling Unseen Categories: When mapping the training averages, any Feature present in the test data that were not in the training data would result in missing values (NaNs). I prevented data loss and leakage by filling these specific features Of testing set with the global mean of training set, ensuring the test set was imputed using only training knowledge.

3-Target Variable Isolation: I ensured that the target variable (is_canceled) and future-looking operational columns (reservation_status) were strictly excluded from all ratio, interaction, and polynomial mathematical formulas.

Task-8

Top 15 Features by Random forest classifier:

Features	Importance
id	0.444011
lead_time_squared	0.052222
previous_cancellations	0.049866
lead_time	0.047103
adr	0.040955
month	0.030398
price_per_person	0.027754
day_of_week	0.026304
special_req_ratio	0.025241
arrival_date_week_number	0.022900
special_requests_squared	0.022325
customer_type_Transient	0.020889
total_of_special_requests	0.020497
total_revenue_potential	0.018635
arrival_date_year	0.016872

Top 15 Features by Mutual info classifier:

Features	Importance_MI
id	0.547821
price_per_person	0.118097
adr	0.113105
total_revenue_potential	0.112044
lead_time	0.092529
lead_time_squared	0.092016
previous_cancellations	0.056024
special_req_ratio	0.046747
special_requests_squared	0.046514
total_of_special_requests	0.046455
required_car_parking_spaces	0.027197
day_of_week	0.025860
booking_changes	0.025021
total_nights	0.021388
days_in_waiting_list	0.019130

```
Redundant Features Dropped
year
customer_type_Transient-Party
special_req_ratio
total_nights
lead_time_squared
special_requests_squared
```

```
Final Selected Features
id
previous_cancellations
lead_time
adr
month
price_per_person
day_of_week
arrival_date_week_number
customer_type_Transient
total_of_special_requests
total_revenue_potential
arrival_date_year
required_car_parking_spaces
stays_in_week_nights
booking_changes
stays_in_weekend_nights
arrival_date_day_of_month
meal_SC
previous_bookings_not_canceled
meal_HB
```

Overlaps and Disagreements:

Both methods strongly agree that *id*, *lead_time* (and its squared version), *adr*, *price_per_person*, *previous_cancellations*, and *special_request_metrics* are the most dominant predictors of cancellation.

They disagree primarily on temporal vs. behavioural features. *Random Forest* prioritised date-based split points like *month*, *arrival_date_week_number*, and *arrival_date_year*. Mutual Information, measuring raw statistical dependence, placed higher importance on financial and physical indicators like *total_revenue_potential*, *booking_changes*, and *required_car_parking_spaces*

Final Selected Feature Set (Top 20):

id, *previous_cancellations*, *lead_time*, *adr*, *month*, *price_per_person*, *day_of_week*, *arrival_date_week_number*, *customer_type_Transient*, *total_of_special_requests*, *total_revenue_potential*, *arrival_date_year*, *required_car_parking_spaces*, *stays_in_week_nights*, *booking_changes*, *stays_in_weekend_nights*, *arrival_date_day_of_month*, *meal_SC*, *previous_bookings_not_canceled*, *meal_HB*

Justification:

This final set was constructed by prioritising the strongest *predictive features* while *explicitly eliminating multicollinearity*. Highly ranked interaction/polynomial features like *lead_time_squared*, *special_requests_squared*, and *total_nights* were dropped because they heavily correlated with their base features, which would otherwise confuse the model.

Table

Version	Features count	Pre-processing	Model	ROC-AUC/F1	Notes
Baseline	30	Simple Imputation + Label Encoding	Logistic Regression	0.7949	High dimensionality, untreated outliers caused instability.
After numeric pre-processing	3	RobustScaler + PowerTransform + Binning	Logistic Regression+ KNN -side experiment	0.6527	Handled extreme outliers in adr and Lead Time; stabilised math distances.
After extraction + construction	37	Polynomials + Ratios + Aggregations + Date Extraction	Logistic Regression	0.6484	Model gained deep behavioural context-but is slightly bloated.
After selection	20	Random Forest Importance + Correlation Filter (>0.85)	Logistic Regression + (Random Forest Classifier + Mutual info Classifier) – prediction only	0.6966	Removed highly correlated redundant features. Fastest training time with focused actionable insights.

Summary

What Mattered Most?

Throughout this Project, it became explicitly clear that data quality and context dictate model success far more than algorithm choice. The raw dataset provided surface-level facts—prices, dates, and headcounts. However, by engaging in feature engineering, we translated these raw facts into behavioural insights. What mattered most was shifting the model's perspective from absolute numbers to relational logic. For example, knowing a room costs \$150 is less valuable than knowing the *price_per_person* is \$75, which instantly segments a luxury solo traveler from a budget-conscious family.

What Changed the Performance Ceiling?

The performance ceiling was drastically raised during two key phases: dealing with outliers and eliminating the curse of dimensionality. The presence of massive outliers in *lead_time* and *adr* severely handicapped distance-based models.

Implementing a *RobustScaler* allowed us to normalise the data without letting extreme edge-cases squash the variance of our normal customers. Furthermore, by utilising a *correlation filter* and *Random Forest importance*, we stripped away redundant dimensions. Removing features that shared >0.85 correlation ensured the model wasn't confused by duplicate signals, raising the accuracy floor and dramatically speeding up computation.